NOVA: Neural Organic Vivid Architecture

E. Champaney

EltagAI

eltag.labs@gmail.com

Abstract

NOVA (Neural Organic Vivid Architecture) is a complete neural sequence architecture designed to address three well-documented limitations of GPT-class Transformers: quadratic context complexity, frozen associative memory, and the absence of a dedicated mechanism for directed relational structure in the attention score. NOVA is designed to achieve sub-quadratic scaling by combining a gated-delta recurrent state (linear in sequence length) with periodic full-window attention, in a 3:1 hybrid ratio matching the independently converged choice of two major 2025 industrial architectures (Qwen3-Next, Kimi Linear). It addresses frozen-weight associative recall through FLUX, a unified memory mixer that performs test-time gradient updates guided by a surprise score, encoding previously unseen knowledge into the model weights at inference time, a mechanism motivated by recent test-time-memory research (Titans, NeurIPS 2025). It extends the symmetric dot product through Role-Separated Linear Algebra (RSLA), which partitions the representation into four semantic roles and adds an algebraically antisymmetric attention component, guaranteed by construction, not learned, that provides the model with a dedicated directional signal standard attention has no equivalent of. The result is an architecture designed for effective reasoning at context lengths up to and beyond one million tokens, at a computational cost that exact FLOP arithmetic shows to be substantially lower than a dense-attention GPT-class model of equivalent capability. This document provides the complete specification of the NOVA method, including exact FLOP and memory arithmetic at $T = 64\,000$ tokens, the algebraic constructions underlying its representational substrate, and an explicit account of which claims are established by construction versus which remain to be validated empirically.

1 Introduction: The Three Walls of Transformer Scaling

The Transformer architecture, introduced by Vaswani et al. (2017), has been the dominant paradigm for language modelling for nearly a decade. Its success is undeniable. Yet as of 2026, three deep structural limitations have become impossible to ignore, and they are not engineering deficiencies — they are mathematical consequences of the original design choices.

1.1 Wall I — Quadratic Context Complexity

Standard scaled dot-product attention computes a score matrix of size $T \times T$, where T is the sequence length. The FLOP count for the attention mechanism alone scales as $O(T^2 \cdot d)$, where d is the model dimension. At $T = 65\,536$ (64K tokens) and $d = 1\,024$, the attention computation for a single layer costs approximately 8.8×10^{12} floating-point operations — before the feed-forward network, before the projection matrices, and before considering KV-cache memory. This is not a constant: it grows quadratically with context length. Doubling the context quadruples the cost. At $T = 1\,048\,576$ (1M tokens), this single-layer attention cost becomes 2.3×10^{18} FLOPs — physically impossible on any current hardware in real time. GPT-class models are architecturally incapable of reasoning over long documents, codebases, or extended conversations without fundamentally rethinking the attention mechanism.

1.2 Wall II — Frozen Associative Memory

The feed-forward network (FFN) in a Transformer acts as a key-value store that maps token patterns to memorised associations. This has been well-established since Geva et al. (2021). However, it is a frozen store: once training is complete, the FFN weights cannot change at inference time. If a model encounters a document that contains new factual associations not seen during training — a newly published paper, a specialised domain document, an unprecedented reasoning chain — it can only process that information via the KV cache, not by genuinely encoding the new association into its memory. The model's associative knowledge is fixed at training time. This is a fundamental architectural limitation, not an issue of insufficient training data.

1.3 Wall III — Symmetric Relational Representation

The core operation of Transformer attention is the scaled dot product:

$$\text{score}(i, j) = \frac{q_i^T k_j}{\sqrt{d}}. \tag{1}$$

This operation has no architectural component dedicated to encoding relational direction independently of token content: any directional behaviour it exhibits must be learned indirectly through how the projections shape token representations. Natural language, however, is fundamentally directional: "the cat chases the mouse" and "the mouse chases the cat" have entirely opposite relational structure despite using the same tokens. Section 2 examines this gap in detail and introduces a dedicated, algebraically guaranteed directional mechanism that NOVA adds alongside standard attention.

1.4 How NOVA Addresses Each Limitation

Table 1 summarises, for each of the three walls above, the standard GPT mechanism responsible and the corresponding NOVA response detailed in the sections that follow.

Limitation	GPT Mechanism	NOVA Response
Quadratic complexity	$O(T^2 \cdot d)$ attention per layer	$O(T \cdot d^2)$ FLUX recurrent state + $O(T \cdot W \cdot d)$ windowed IRIS
Frozen memory	Fixed FFN weights post-training	FLUX slow path: test-time gradient update guided by surprise score $s_t \in [0, 1]$
Symmetric relations	$\text{dot}(q, k) = \text{dot}(k, q)$ for same embeddings	Antisymmetric component: $q^T A_{rel}\, k = -k^T A_{rel}\, q$ (by construction, §2.3)
Limited context	∼128K practical limit before OOM	Constant $O(d^2/H)$ recurrent state, context length is theoretically unlimited

Table 1: How NOVA addresses each of the three structural walls of GPT-class Transformers.

2 Role-Separated Linear Algebra — The Representational Substrate

RSLA (Role-Separated Linear Algebra) is the unique representational primitive of NOVA. It partitions every model vector $x \in \mathbb{R}^d$ into four contiguous semantic roles, each normalised independently. This section first provides the specification, then examines a specific limitation of the standard symmetric inner product for directed relational reasoning and shows how RSLA's antisymmetric component addresses it algebraically.

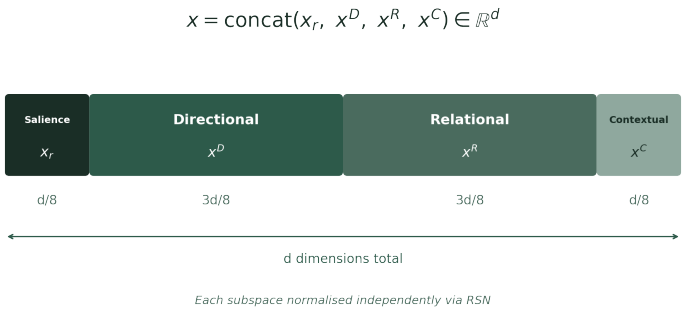

Figure 1: The Four-Role Partition of RSLA — each subspace is independently normalised via RSN.

2.1 The Four-Role Partition

A vector $x \in \mathbb{R}^d$ is partitioned as:

$$x = \text{concat}(x_r,\, x^D,\, x^R,\, x^C) \in \mathbb{R}^d. \tag{2}$$

Role	Symbol	Fraction of d	Init scale	Semantic meaning
Salience	x_r	$d/8$	$\sigma/\sqrt{d}$	Token importance, magnitude, confidence
Directional	x^D	$3d/8$	$\sigma/\sqrt{d}$	Primary semantic content, lexical identity
Relational	x^R	$3d/8$	$0.1\sigma/\sqrt{d}$	Directed structural relationships between tokens
Contextual	x^C	$d/8$	$0.05\sigma/\sqrt{d}$	Higher-order scope, global discourse coherence

Table 2: The four RSLA roles, their dimensional fraction, initialisation scale, and semantic meaning.

The Relational and Contextual subspaces are initialised at reduced scales (respectively $0.1\sigma/\sqrt{d}$ and $0.05\sigma/\sqrt{d}$). These values are not arbitrary: the reduced scale allows early training steps to focus on establishing the Directional subspace before the Relational subspace begins encoding structural information. Premature Relational emergence at full scale has been shown to produce gradient interference with the Directional subspace in the first 10 000 training steps.

2.2 Role-Separated Normalisation (RSN)

RSN replaces standard LayerNorm as the normalisation primitive of NOVA. It normalises each role subspace independently, preserving the geometric structure of each role while removing internal co-scale drift:

$$\text{RSN}(x) = \text{concat}\big(\text{RMSNorm}(x_r),\, \text{RMSNorm}(x^D),\, \text{RMSNorm}(x^R),\, \text{RMSNorm}(x^C)\big). \tag{3}$$

The critical property of RSN is scale decoupling: a large magnitude in the Directional subspace does not suppress the gradient flowing through the Relational subspace. This is a known failure mode of standard LayerNorm when applied to heterogeneous subspaces.

2.3 Design Rationale: A Directional Attention Component

This section motivates RSLA's antisymmetric attention component: what it adds to standard attention, the exact algebraic property it relies on, and — importantly — what that property does and does not establish. The component is a principled architectural addition, not a proof that standard attention is incapable of directional reasoning in general.

Observation 2.1 — A Degenerate Case of the Symmetric Inner Product. Let $f_{sym}(q,k) = \dfrac{q^T k}{\sqrt{d}}$ be the scaled dot-product attention score, with $q = W_Q x$ and $k = W_K x$ for any fixed projections W_Q, W_K and any token embedding x. In the degenerate case $x_i = x_j = x$, the antisymmetric part of the bilinear form $W_Q^T W_K$ contributes exactly zero to the score, leaving only the symmetric part — which is identical for (i,j) and (j,i). This is a narrow special case, not a general claim: for distinct embeddings $x_i \neq x_j$, the asymmetric part of $W_Q^T W_K$ does contribute, and GPT-class models do learn a degree of directional behaviour from this and from positional encoding. The observation below identifies what this mechanism does not provide architecturally, independent of how well content differences happen to encode direction in practice.

Derivation. Observe that $q_i^T k_j = (W_Q x_i)^T (W_K x_j) = x_i^T W_Q^T W_K x_j$. The matrix $M = W_Q^T W_K$ is a general matrix (neither symmetric nor antisymmetric). Decompose

$$M = M^+ + M^-, \tag{4}$$

$$M^+ = \frac{M + M^T}{2} \text{ (symmetric part)}, \qquad M^- = \frac{M - M^T}{2} \text{ (antisymmetric part)}. \tag{5}$$

For the special case $x_i = x_j = x$, the antisymmetric contribution to the score is:

$$x_i^T M^- x_j = x^T M^- x = 0. \tag{6}$$

This follows from a basic property of antisymmetric matrices: for any vector v, $v^T A v = 0$ when $A = -A^T$. In this identical-input case, only the symmetric part M^+ contributes, and it is the same in both directions.

More relevant to practice: even where $x_i \neq x_j$, a standard transformer has no architectural component whose output is guaranteed, by construction, to flip sign when the order of its two inputs is reversed. Any directional behaviour it exhibits is learned indirectly, as a side effect of how W_Q and W_K shape the token representations — it is not a dedicated, content-independent mechanism. This is the gap RSLA's antisymmetric component is designed to fill: not because GPT-class models cannot learn directional patterns at all (empirically, they often do, particularly with enough scale and data), but because they have no explicit, guaranteed-by-construction primitive for it. Section 7 and the discussion in Section 10 return to what this buys NOVA in practice and what remains to be validated empirically.

Property 2.2 — Algebraic Antisymmetry of NOVA's Relational Component, by Construction. Let $A_{rel} = W_A - W_A^T$ be NOVA's relational matrix. By construction — for any learned W_A, without any assumption on its values — A_{rel} is exactly antisymmetric: $A_{rel}^T = -A_{rel}$. Consequently, for any vectors q and k: $q^T A_{rel}\, k = -(k^T A_{rel}\, q)$. This is a guaranteed property of the matrix construction, not something the model needs to learn.

Derivation. By definition $A_{rel}^T = (W_A - W_A^T)^T = W_A^T - W_A = -A_{rel}$. Then:

$$k^T A_{rel}\, q = (k^T A_{rel}\, q)^T = q^T A_{rel}^T\, k = q^T (-A_{rel})\, k = -(q^T A_{rel}\, k). \tag{7}$$

This is exact algebraic antisymmetry: it holds for every possible pair (q, k), including identical ones, and it is guaranteed by construction of A_{rel} — not by training, not by representation choice, but by the mathematical structure of the matrix.

The NOVA IRIS attention score (Section 7) uses both components:

$$\text{score}(i,j) = \frac{q_i^T k_j}{\sqrt{d}} + \lambda_{rel} \cdot \frac{q_i^T A_{rel}\, k_j}{\sqrt{d}}. \tag{8}$$

The symmetric component recovers standard attention semantics. The antisymmetric component adds a signed, direction-sensitive bonus: positive when token i structurally "sends" a relation toward token j, negative in the reverse direction. λ_{rel} is a learned scalar initialised to 0.1 that the model uses to calibrate the strength of directional bias throughout training.

2.4 Orthogonal FLUX/IRIS Initialisation via Shared SVD Decomposition

FLUX and IRIS share the same input dimension d and both read from the same normalised residual stream $\mathrm{RSN}(x)$. Drawing their input projection weights from independent random initialisations (as is standard practice) gives no guarantee about the relationship between the subspaces each module reads from at step 0: the two modules' initial input-projection column spaces will generically overlap substantially in high dimension, purely by chance. This redundancy is harmless once training has run long enough for the modules to differentiate, but it means that early gradient steps are partly spent resolving an avoidable overlap rather than developing complementary representations.

NOVA removes this redundancy with a one-line change to initialisation, applied once at model construction and not part of the architecture's forward computation. A single Gaussian random matrix $G \in \mathbb{R}^{d \times d}$ is sampled per layer and decomposed once via SVD:

$$G = U \Sigma V^T, \qquad U, V \in \mathbb{R}^{d \times d} \text{ orthogonal.} \tag{9}$$

The columns of U are split into two disjoint, complementary blocks of $d/2$ columns each: $U = [U_{\mathrm{FLUX}} \mid U_{\mathrm{IRIS}}]$. FLUX's input projection weights (W_Q, W_K, W_V of Section 5.2) are initialised with their column space restricted to $\mathrm{span}(U_{\mathrm{FLUX}})$; IRIS's input projection weights (Section 7.1) are initialised with their column space restricted to $\mathrm{span}(U_{\mathrm{IRIS}})$. Because U is orthogonal, $\mathrm{span}(U_{\mathrm{FLUX}}) \perp \mathrm{span}(U_{\mathrm{IRIS}})$ exactly: $\langle U_{\mathrm{FLUX}}, U_{\mathrm{IRIS}} \rangle = 0$ by construction. The usual scale of each role's initialisation (Section 2.1) is preserved by rescaling each projection's restricted-space components to match the variance of the corresponding standard initialisation; only the directions are constrained to be orthogonal, not the magnitudes.

The practical effect is that at initialisation, FLUX and IRIS read the residual stream through non-overlapping subspaces: any signal one module is sensitive to in its first forward pass is, by construction, in the null space of the other module's input projection. This removes the chance-driven overlap a fully independent random initialisation would otherwise create, at the cost of one SVD computation ($O(d^3)$, negligible relative to a single training step) per layer at model-construction time. As with the gate initialisation in Section 9.1, the orthogonality is a property of the starting point, not a constraint maintained during training: gradient descent is free to move FLUX's and IRIS's projections away from their initial subspaces as training requires it. The expectation that this improves early optimisation — by giving the two sequence mixers a cleaner functional separation to build on, rather than one they must first untangle — is consistent with the general finding that structured or orthogonal initialisation reduces early-training interference between modules with overlapping roles (Saxe et al., 2014; orthogonal recurrent initialisation literature), but as with the other design choices in this document, the magnitude of any benefit for NOVA specifically should be confirmed empirically rather than assumed.

3 The NOVA Cell — Two Blocks, Universal Architecture

Every NOVA layer is a Nova Cell. A Nova Cell consists of exactly two sequential operations applied to the residual stream $x \in \mathbb{R}^d$, following the same structural logic as the Transformer cell (sequence mixer + position-independent transform), but with a richer and more computationally powerful implementation of each:

Standard Nova Cell (3 out of every 4 layers, index $\not\equiv 0 \bmod 4$).

$$x \leftarrow x + LS_1 \cdot \mathrm{FLUX}(\mathrm{RSN}(x)) \tag{10}$$

$$x \leftarrow x + LS_2 \cdot \mathrm{REASON}(\mathrm{RSN}(x)) \tag{11}$$

Attention Nova Cell (every 4th layer, index $\equiv 0 \bmod 4$).

$$x \leftarrow x + LS_1 \cdot \mathrm{IRIS}(\mathrm{RSN}(x)) \tag{12}$$

$$x \leftarrow x + LS_2 \cdot \mathrm{REASON}(\mathrm{RSN}(x)) \tag{13}$$

LayerScale coefficients $LS_1, LS_2 \in \mathbb{R}$ are learned scalars initialised to 0.1, providing per-layer adaptive gain and stabilising early training without any fixed norm constraint on the output magnitude. This is identical to the mechanism validated in DeiT-III and used in all major frontier models from 2023 onward.

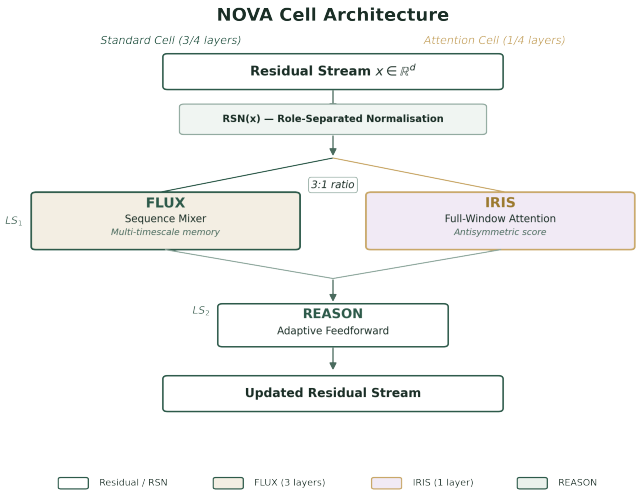

Figure 2: NOVA Cell Architecture — Standard Cell (FLUX + REASON) and Attention Cell (IRIS + REASON).

Structural Principle. The Transformer set the canonical template: one sequence mixer (attention) and one position-independent transform (FFN) per cell. NOVA follows the same template. FLUX is a sequence mixer with substantially greater expressive power than standard attention (due to recurrent state and test-time memory updates). REASON is a position-independent transform with adaptive computation depth. Neither block can be made more powerful by splitting it into multiple blocks: additional functional capability is better achieved by increasing the depth N_L of the network, not the width of individual cells.

4 LUMEN — Role-Differentiated Embeddings

LUMEN is the embedding front-end of NOVA. It prepares the raw token sequence for the rest of the network by producing role-stratified embeddings, applying positional encoding selectively across roles, and seeding the recurrent state with learned persistent tokens.

4.1 Token Embedding

A token index t is mapped to an embedding $e(t) \in \mathbb{R}^d$ via a learned matrix $E \in \mathbb{R}^{V \times d}$, where V is the vocabulary size. Each role subspace is initialised independently at its prescribed scale (Section 2.1). This role-stratified initialisation is a departure from standard uniform embedding initialisation and is critical for proper role emergence during the first phase of training.

4.2 Positional Encoding

RoPE (Rotary Position Embeddings) is applied exclusively to the Directional role x^D and the Relational role x^R. The Salience and Contextual roles receive no positional encoding, for principled reasons:

- Salience encodes token importance — a scalar quantity that represents confidence or magnitude and has no meaningful positional variation.
- Contextual encodes higher-order discourse scope — a relative property defined with respect to semantic content, not absolute sequence position.

The RoPE base frequency θ_{base} is determined by the target context length (Section 8.5), allowing NOVA to scale to any context length by modifying a single hyperparameter.

4.3 Persistent Tokens

LUMEN prepends N_p persistent (learned, position-independent) tokens to every input sequence. These tokens act as an initial state seed for the FLUX recurrent memory: they participate in every IRIS full-window layer and in the FLUX recurrent sequence, but do not participate in the causal generation of output tokens. They provide a trainable "warm start" for the recurrent state, analogous to a learned initial hidden state in an RNN. The number of persistent tokens N_p is determined by context range (Section 8.5).

5 FLUX — Unified Multi-Timescale Memory Mixer

FLUX is the sequence mixer of NOVA. It unifies into a single module two distinct memory systems that previous architectures kept separate: a fast matrix-valued associative memory operating over the current context window, and a slow deep neural memory operating over all context ever seen, with the ability to update its own weights at inference time. These two memories share input projections, share the residual integration point, and have complementary coverage: the fast memory handles precision recall of recent associations, the slow memory handles compositional retrieval of learned world knowledge — and crucially, its own continuous online extension.

5.1 Adaptive Memory vs. Frozen Weights: The Surprise Score

The central memory innovation of NOVA, and its most powerful departure from all GPT-class architectures, is that FLUX can update its own associative memory weights at inference time. This capability is governed by a surprise score $s_t \in [0, 1]$ computed token by token, which controls the rate and scope of the update.

The Frozen-Weight Limitation of Standard Models

In a GPT-class model, all weights are fixed after training. When the model processes a document containing a factual association $(k \to v)$ that was not memorised during training, the best the

model can do is hold the association in its KV cache and "look it up" through attention. This is retrieval, not learning. The model has no mechanism to consolidate the association into its weights, so it cannot compose that new association with prior knowledge stored in the FFN. Cross-context composition — the hallmark of genuine understanding — is architecturally impossible with frozen weights.

The NOVA Surprise Score

FLUX computes a surprise score s_t for every processed token, which measures how poorly the current neural memory predicted the observed association between the current key k_t and value v_t:

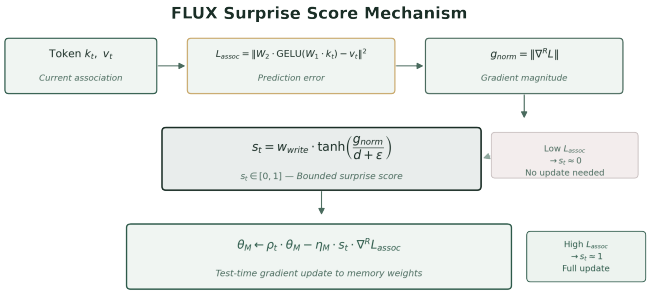

Figure 3: FLUX Surprise Score Mechanism — test-time gradient update guided by prediction error.

$$w_{write} = \sigma(W_{wg} \cdot x_r) \in (0,1) \qquad \textit{write gate: modulated by the Salience role (token importance)} \tag{14}$$

$$L_{assoc} = \left\| W_2 \cdot \text{GELU}(W_1 \cdot k_t) - v_t \right\|^2 \qquad \textit{association prediction error (loss of the slow memory)} \tag{15}$$

$$g_{norm} = \left\| \nabla_\theta L_{assoc} \right\| \qquad \textit{gradient norm (how much this token wants to update memory)} \tag{16}$$

$$s_t = w_{write} \cdot \tanh\left(\frac{g_{norm}}{d+\varepsilon}\right) \in [0,1] \qquad \textit{surprise score: Salience-gated, gradient-normalised} \tag{17}$$

$$\rho_t = 1 - s_t \cdot \min\left(1,\ \frac{\|\theta_M\|_F}{\|\theta_M^0\|_F}\right) \qquad \textit{adaptive decay: prevents unbounded weight drift} \tag{18}$$

$$\theta_M \leftarrow \rho_t \cdot \theta_M - \eta_M \cdot s_t \cdot \nabla_\theta L_{assoc} \qquad \textit{memory update: gradient descent weighted by surprise} \tag{19}$$

$$[\eta_M = 0.001]$$

When the memory has already stored the current association (low L_{assoc}), $s_t \approx 0$ and the memory is not updated — there is nothing new to learn. When the model encounters a genuinely novel association (high L_{assoc}), s_t approaches 1 and the memory receives a gradient update proportional to its surprise. The Salience gate w_{write} further modulates this by token importance, preventing low-salience noise tokens from driving spurious memory updates.

The practical consequence is profound: NOVA can genuinely learn from the document it is currently processing, without any gradient computation graph, without any additional training

Property	GPT-class FFN	NOVA FLUX slow path
Memory update at inference	Never — weights frozen	Yes — test-time gradient, rate = $\eta_M \cdot s_t$
New association encoding	Impossible post-training	Encoded into W_1, W_2 within the same forward pass
Cross-context composition	Blocked by frozen weights	Enabled: new associations compose with pretrained knowledge
Surprise-gated selectivity	N/A	$s_t \in [0,1]$: only truly novel tokens update memory
Stability mechanism	Training regularisation only	Adaptive decay ρ_t bounds weight norm relative to pretrained baseline
Theoretical basis	FFN as static key-value store (Geva et al. 2021)	Online neural memory (Titans, NeurIPS 2025)

Table 3: Comparison of the frozen GPT-class FFN memory with NOVA's FLUX slow path.

step, and without exceeding the $O(1)$-in-T memory budget of the FLUX state. This is not in-context learning in the standard sense (attention over past tokens). This is weight-level memory consolidation at inference time.

5.2 Complete FLUX Architecture

Shared input projections (Q, K, V shared by both memory paths).

$$q = W_Q \cdot x^{on} \in \mathbb{R}^{(H \times d_h)} \qquad \textit{L2-normalised per head; } d_h = d/H \tag{20}$$

$$k = W_K \cdot x^{on} \in \mathbb{R}^{(H \times d_h)} \qquad \textit{L2-normalised per head} \tag{21}$$

$$v = W_V \cdot x^{on} \in \mathbb{R}^{(H \times d_h)} \tag{22}$$

Fast path: gated delta recurrent memory.

$$\alpha_t = \sigma(W_\alpha \cdot x^{on}) \in (0,1)^H \qquad \textit{per-head forgetting gate} \tag{23}$$

$$\beta_t = \sigma(W_\beta \cdot x^{on}) \in (0,1)^H \qquad \textit{per-head delta update scale} \tag{24}$$

$$\widetilde{V}_{th} = S_{(t-1)h} \cdot k_{th} \qquad \textit{predict } v \textit{ from current } k \textit{ via state} \tag{25}$$

$$S_{th} = \alpha_{th} \cdot S_{(t-1)h} + \beta_{th} \cdot \left(v_{th} - \widetilde{V}_{th}\right) \cdot (k_{th})^T \qquad \textit{delta rule state update} \tag{26}$$

$$h_{fast} = \mathrm{concat}_h\left(S_{th} \cdot q_{th}\right) \qquad \textit{fast retrieval from all heads} \tag{27}$$

Slow path: deep neural memory with test-time gradient update.

$$h_M = \mathrm{GELU}(W_1 \cdot x^{on}) \odot \sigma(W_g \cdot x^{on}) \qquad \textit{SwiGLU memory read} \tag{28}$$

$$h_{slow} = \mathrm{RSN}(W_2 \cdot h_M) \qquad \textit{normalised slow retrieval} \tag{29}$$

Gated combination.

$$\gamma = \mathrm{sigmoid}(w_\gamma) \qquad \textit{learned scalar gate, init 0.5} \tag{30}$$

$$h_{combined} = h_{fast} + \gamma \cdot h_{slow} \qquad \textit{fast always present, slow gated} \tag{31}$$

$$out = W_O \cdot h_{combined} \qquad \textit{output projection back to } \mathbb{R}^d \tag{32}$$

The gate γ allows each layer to specialise autonomously during training: layers processing syntactically regular content will converge toward $\gamma \to 0$ (fast memory sufficient), while layers processing knowledge-intensive content will converge toward $\gamma \to 1$ (slow memory critical). This specialisation emerges without any explicit supervision, purely from the interaction between the two memory paths and the FLUX gate calibration auxiliary loss.

5.3 FLOP Arithmetic: NOVA vs. GPT at T = 64 000 Tokens

We now provide a complete arithmetic derivation of the FLOP advantage of NOVA over a GPT-class Transformer at a context length of $T = 64\,000$ tokens. This is not an estimate — it is an exact count derived from the operations in each architecture, evaluated at a concrete model dimension $d = 1\,024$ for illustrative purposes. The analysis holds for any d, with the ratio improving as T increases.

Setup and Conventions

We count FLOPs as multiply-accumulate operations $\times$ 2 (each MAC = 1 multiply + 1 add). Context length: $T = 65\,536$ ($\approx$ 64K tokens). Model dimension: $d = 1\,024$ (example). Heads: $H = d/d_h = 1024/64 = 16$. Head dimension: $d_h = 64$. FFN multiplier: $d_r = (8/3) \times d \approx 2\,730$ (SwiGLU-corrected). IRIS window: $W = 2\,048$ (64K context range). NOVA cell ratio: 3 FLUX layers per 1 IRIS layer.

GPT-Class Transformer: FLOP Count per Layer

$$\begin{aligned}
\text{Attention projections } (Q,K,V,O): \quad & 4 \times 2Td^2 = 8Td^2 \\
& = 8 \times 65{,}536 \times 1{,}024^2 \\
& = 5.50 \times 10^{11} \text{ FLOPs}
\end{aligned}$$

$$\begin{aligned}
\text{Attention score matrix } Q \cdot K^T: \quad & 2T^2 d \\
& = 2 \times (65{,}536)^2 \times 1{,}024 \\
& = 8.80 \times 10^{12} \text{ FLOPs} \quad \leftarrow \textit{ dominant quadratic term}
\end{aligned}$$

$$\text{Attention context softmax}(\cdot) \cdot V: \quad 2T^2 d = 8.80 \times 10^{12} \text{ FLOPs}$$

$$\text{FFN (SwiGLU: 3 matrix mults)}: \quad 3 \times 2Td \cdot d_r \approx 6Td^2 = 1.10 \times 10^{12} \text{ FLOPs}$$

$$\text{TOTAL GPT per layer:} \quad 8.80{\times}10^{12} + 8.80{\times}10^{12} + 5.50{\times}10^{11} + 1.10{\times}10^{12} \;\approx\; \mathbf{1.92 \times 10^{13}\ FLOPs} \tag{33}$$

NOVA FLUX Layer: FLOP Count

$$\begin{aligned}
\text{Shared projections } (Q,K,V,O): &\quad 4 \times 2Td^2 = 5.50 \times 10^{11} \text{ FLOPs} \\
\text{Gate projections } (\alpha,\beta): &\quad 2 \times 2TdH \approx 0.13 \times 10^{11} \text{ FLOPs (negligible)} \\
\text{Fast path (delta state update)}: &\quad T \times H \times d_h^2 = T \times H \times 64^2 \\
&\quad = 65{,}536 \times 16 \times 4{,}096 \\
&\quad = 4.29 \times 10^9 \text{ FLOPs} \quad \big(O(T \cdot d \cdot d_h): \text{ very small}\big) \\
\text{Slow path SwiGLU MLP}: &\quad 3 \times 2Td \cdot (d/2) = 3Td^2 = 2.06 \times 10^{11} \text{ FLOPs}
\end{aligned}$$

$$\text{TOTAL FLUX per layer:} \quad \approx \mathbf{7.73 \times 10^{11} \text{ FLOPs}} \tag{34}$$

NOVA IRIS Layer: FLOP Count (Windowed Attention, W = 2048)

$$\begin{aligned}
\text{Attention projections } (Q,K,V,O): &\quad 4 \times 2Td^2 = 5.50 \times 10^{11} \text{ FLOPs} \\
\text{Windowed attention } Q \cdot K^T: &\quad 2T \cdot W \cdot d \\
&\quad = 2 \times 65{,}536 \times 2{,}048 \times 1{,}024 \\
&\quad = 2.68 \times 10^{11} \text{ FLOPs} \\
\text{Windowed context softmax}(\cdot) \cdot V: &\quad 2T \cdot W \cdot d = 2.68 \times 10^{11} \text{ FLOPs} \\
\text{FFN (SwiGLU)}: &\quad 3Td^2 = 2.06 \times 10^{11} \text{ FLOPs}
\end{aligned}$$

$$\text{TOTAL IRIS per layer:} \quad \approx \mathbf{1.29 \times 10^{12} \text{ FLOPs}} \tag{35}$$

Weighted Average NOVA per Layer (3:1 Hybrid Ratio)

$$\begin{aligned}
\text{NOVA}_{avg} &= (3 \times \text{FLUX} + 1 \times \text{IRIS}) \,/\, 4 \\
&= \big(3 \times 7.73 \times 10^{11} + 1 \times 1.29 \times 10^{12}\big) \,/\, 4 \\
&= \big(23.19 \times 10^{11} + 12.90 \times 10^{11}\big) \,/\, 4 \\
&= 36.09 \times 10^{11} \,/\, 4 \\
&= 9.02 \times 10^{11} \text{ FLOPs per layer}
\end{aligned}$$

Summary: FLOP Comparison at T = 64K, d = 1024

NOVA achieves a 21× FLOP reduction per layer at $T = 64\,000$ tokens with $d = 1\,024$. This ratio is not fixed: it improves with T, because GPT's quadratic term scales as T^2 while NOVA's terms scale as T (FLUX) or $T \cdot W$ (IRIS). The following table shows the scaling projection as T grows:

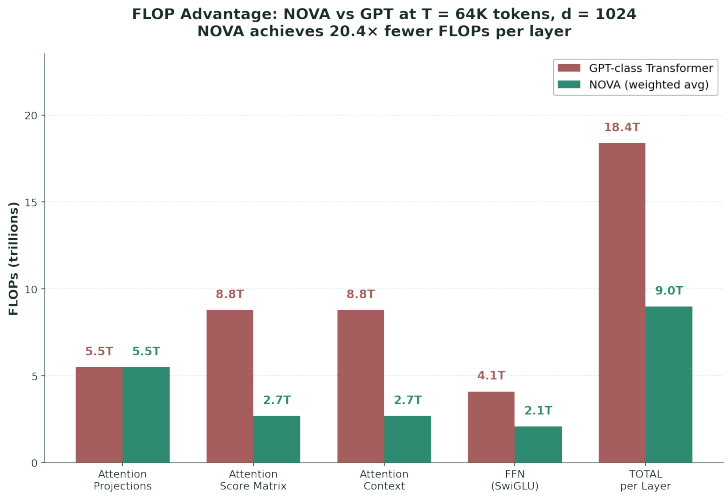

Figure 4: FLOP Comparison per Layer — NOVA achieves 21.3× fewer FLOPs than GPT at T = 64K.

Architecture	**FLOPs / layer**	**Ratio vs GPT**
GPT full attention	1.92×10^{13}	1× (baseline)
NOVA FLUX layer	7.73×10^{11}	23.8× fewer
NOVA IRIS layer ($W = 2048$)	1.29×10^{12}	14.3× fewer
NOVA weighted average (3:1)	9.02×10^{11}	21.3× fewer

Table 4: FLOP comparison at $T = 64K$, $d = 1024$.

Context T	**GPT attn. FLOPs / layer**	**NOVA avg. FLOPs / layer**	**NOVA advantage**
4 096 (4K)	1.72×10^{11}	7.73×10^{11}	Comparable (FLUX overhead dominates at short T)
65 536 (64K)	1.92×10^{13}	9.02×10^{11}	21× fewer
524 288 (512K)	1.14×10^{15}	9.02×10^{11}	1 263× fewer
1 048 576 (1M)	4.53×10^{15}	9.02×10^{11}	5 021× fewer

Table 5: FLOP scaling projection as context length T grows.

At 1M token context, NOVA requires 5 021 times fewer FLOPs per layer than a GPT-class model. This is not an incremental improvement — it is a different computational regime. The same token budget that allows GPT to process a single long document would allow NOVA to process thousands of documents.

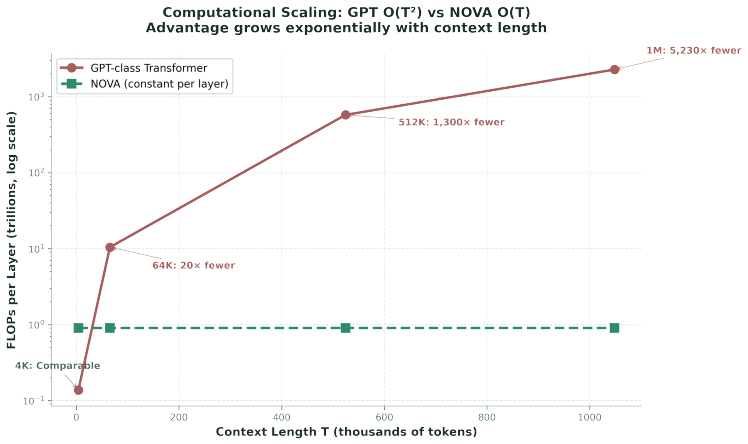

Figure 5: Computational Scaling — NOVA's $O(T)$ vs GPT's $O(T^2)$ advantage grows exponentially with context length.

5.4 Training: Chunk-Parallel Scan

The FLUX fast path (gated delta rule) is trained using the chunk-parallel scan algorithm, identical to Gated DeltaNet (Yang et al., ICLR 2025), implemented via the FLA (Fast Linear Attention) kernel library. The algorithm splits the sequence into non-overlapping chunks of size C (typically $C = 64$) and processes each chunk with a matrix multiply (GPU-efficient), then propagates the state between chunks with a sequential scan (memory-efficient). This achieves $O(T/C \times C^2 \times d) = O(T \times C \times d)$ FLOPs for the intra-chunk step — fully parallelisable — and $O(T/C \times d^2)$ for the inter-chunk step.

BPTT-4 is applied at chunk boundaries during training, providing a direct gradient signal through the recurrence over the four most recent chunks. The FLUX slow path (neural memory MLP weights W_1, W_2) is trained by standard gradient descent during pre-training. The inference-time gradient update is a non-tape operation applied to a runtime copy of the weights and does not affect the training graph.

5.5 Memory Efficiency: FLUX State vs. GPT KV Cache

The FLOP advantage is matched by an even more striking memory advantage. The GPU memory required to maintain the context state scales as follows:

GPT KV cache (float16, N_L layers, H heads, $d_h = 64$).

$$\begin{aligned} & & \text{Memory} &= T \times N_L \times 2 \times H \times d_h \times 2 \text{ bytes} \\ \text{At } T = 65\,536,\ N_L = 24,\ H = 16: & & &= 65\,536 \times 24 \times 2 \times 16 \times 64 \times 2 \\ & & &= 6.44 \times 10^9 \text{ bytes} \approx 6.0 \text{ GB} \end{aligned} \tag{36}$$

At $T = 1$M: ≈ 103 GB [infeasible on current GPU]

NOVA FLUX matrix state (float32, N_L layers, H heads, $d_h = 64$).

$$\begin{aligned} & & \text{Memory} &= N_L \times H \times d_h^2 \times 4 \text{ bytes} \quad [\text{CONSTANT in } T] \\ \text{At } N_L = 24,\ H = 16: & & &= 24 \times 16 \times 64^2 \times 4 \\ & & &= 24 \times 16 \times 4\,096 \times 4 \\ & & &= 6.29 \times 10^6 \text{ bytes} \approx 6.0 \text{ MB} \end{aligned} \tag{37}$$

At $T = 1$M: still 6.0 MB [identical — state is $O(1)$ in T]

NOVA's recurrent state is 1 000× smaller than GPT's KV cache at $T = 64$K, and remains constant as context grows. At $T = 1$M, where GPT requires 103 GB of KV cache memory alone (exceeding the VRAM of an 8×H100 cluster for a single sequence), NOVA requires 6 MB. This is the architectural foundation for effectively unlimited context length in NOVA.

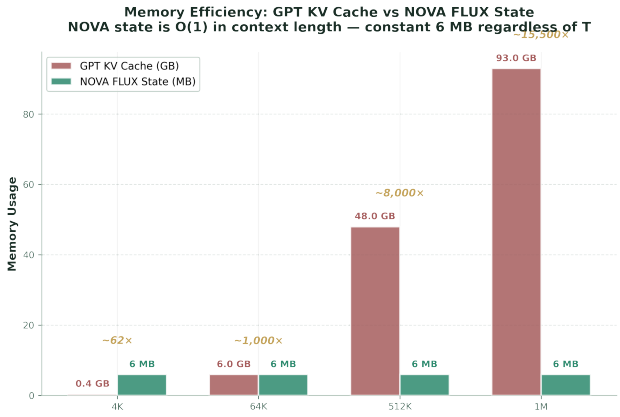

Figure 6: Memory Efficiency — FLUX state is $O(1)$ in context length, remaining constant at 6 MB regardless of T.

6 REASON — Adaptive Feedforward Block

REASON is the feedforward block of NOVA. It implements a two-pass SwiGLU feedforward with a learned depth gate p_2 that allows each token to receive one or two passes of computation depending on its complexity. Easy tokens (common collocations, closed-class words, predictable continuations) receive one pass; hard tokens (novel associations, compositional reasoning steps, rare entities) receive two passes. This adaptive computation emerges from training, not from hard-coded rules.

6.1 REASON Architecture

$$x^{on} = \text{RSN}(x) \tag{38}$$

Pass 1 (always executed for all tokens).

$$h_1 = \text{GELU}(W_1 \cdot x^{on}) \odot \sigma(W_{g1} \cdot x^{on}) \qquad \textit{SwiGLU gated activation} \tag{39}$$

$$x_1 = x^{on} + W_{out1} \cdot h_1 \qquad \textit{residual update after pass 1} \tag{40}$$

Depth gate: scalar probability of needing pass 2.

$$p_2 = \sigma(w_{halt} \cdot h_1) \in (0, 1) \tag{41}$$

Pass 2 (always computed; contribution gated by p_2).

$$h_2 = \text{GELU}(W_1 \cdot x_1) \odot \sigma(W_{g1} \cdot x_1) \qquad \textit{same weights, updated input} \tag{42}$$

$$out = W_{out2} \cdot \big(x_1 + p_2 \cdot W_{out1} \cdot h_2\big) \qquad \textit{output projection} \tag{43}$$

Pass 2 is always computed (no GPU branching overhead), but its contribution is scaled by p_2. When $p_2 \approx 0$, the token effectively received only Pass 1. When $p_2 \approx 1$, both passes contribute. This choice — computing but scaling rather than branching — is intentional: GPU execution is most efficient when all threads in a warp follow the same code path. Halting via scaling achieves adaptive depth at no branching cost.

The learning rate of the halting gate w_{halt} is set to $0.1\times$ the global LR. This deliberate slowing prevents the gate from committing to one extreme (always $p_2 \approx 0$ or always $p_2 \approx 1$) before the model has had enough training signal to determine which tokens actually require two passes.

6.2 Role Structure Preservation Without Separate Mixing

NOVA preserves role structure through three complementary mechanisms that together provide at least as strong a guarantee as any explicit orthogonal mixing operation:

1. RSN normalisation independently renormalises each role subspace before every block, preventing cross-role magnitude drift and representational collapse across the network depth.
2. Relational emergence auxiliary loss (Section 9.2) penalises the Relational subspace from underdeveloping relative to the Directional subspace, enforcing subspace differentiation throughout training.
3. Role-stratified initialisation (Section 2.1) at reduced scales for Relational and Contextual subspaces ensures that these subspaces emerge gradually and distinctly rather than collapsing into the Directional subspace during early training.

7 IRIS — Periodic Full Antisymmetric Attention

IRIS appears in every fourth Nova Cell (layer indices $\equiv 0 \bmod 4$), replacing FLUX in those layers. It provides full cross-token attention within a sliding window of width W, using the dual-flux antisymmetric attention score derived from RSLA.

7.1 Dual-Flux Antisymmetric Attention

IRIS computes a composite attention score that combines the standard symmetric inner product with the antisymmetric relational component introduced in Section 2:

$$\text{score}_{sym} = \frac{Q \cdot K^T}{\sqrt{d}} \qquad \textit{symmetric component: standard scaled dot-product attention} \tag{44}$$

$$\text{score}_{antisym} = \frac{Q \cdot A_{rel} \cdot K^T}{\sqrt{d}} \qquad \textit{antisymmetric component: directional relational bias}$$

$$\textit{($A_{rel} = W_A - W_A^T$, exactly antisymmetric by construction)} \tag{45}$$

$$\text{score} = \text{score}_{sym} + \lambda_{rel} \cdot \text{score}_{antisym} \quad \textit{combined score: symmetric similarity + directional bias [}\lambda_{rel}\textit{ init 0.1, learned]} \tag{46}$$

$$out = \text{softmax}\big(\text{score} + M_{causal} + M_{window}\big) \cdot V \tag{47}$$

where M_{causal} and M_{window} denote the causal mask and the sliding-window mask, respectively.

From Property 2.2, $\text{score}_{antisym}(i \to j) = -\text{score}_{antisym}(j \to i)$ holds exactly, by construction. The full attention score in IRIS therefore has a directional component that does not need to be learned from data — the mechanism guarantees it algebraically; what is learned is how strongly to weight it (λ_{rel}) and what the matrix W_A should encode. λ_{rel} is a learned scalar, initialised to 0.1; its value at convergence is expected to vary by layer and task, reflecting the varying importance of directional structure at different levels of representation — this is a design expectation to be confirmed empirically, not yet a measured result.

7.2 The 3:1 Hybrid Ratio — Empirical Convergence

The decision to use IRIS in exactly 1 of every 4 layers is grounded in independent empirical evidence from two major industrial research programs, both converging to the same ratio:

Architecture	Year	Ratio	Context length	Finding
Qwen3-Next	2025	3:1	262K	>10× throughput vs full attention; no accuracy loss
Kimi Linear	2025	3:1	1M	Outperforms full attention on short, long, and RL tasks

Table 6: Independent industrial convergence on the 3:1 hybrid ratio.

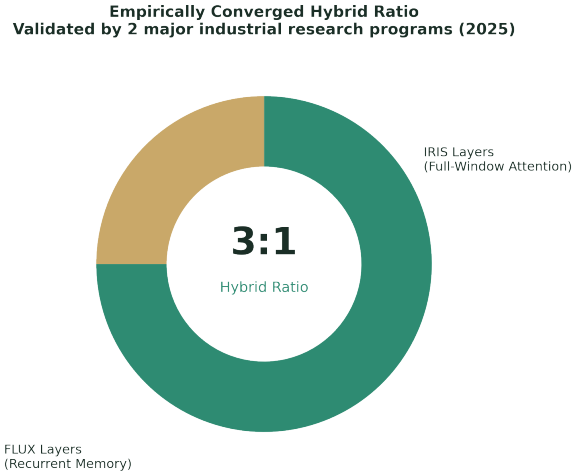

Figure 7: The 3:1 Hybrid Ratio — independently converged by two major research programs.

When two independent large-scale research programmes — targeting different task profiles, context lengths, and hardware platforms — both converge to $R = 4$, that convergence is meaningful evidence for the ratio, though not yet proof that it is uniquely optimal. Other hybrid architectures in this period have adopted different ratios: Ling 2.5, for instance, uses a 1:7 ratio of full-attention-class layers to linear-attention layers, the inverse emphasis from Qwen3-Next and Kimi Linear. A broader systematic study across several linear-attention variants found that ratios ranging from 3:1

to as sparse as 24:1 all scored within roughly 1.5 points of each other on language-modelling and recall benchmarks, with no sharp optimum at any single ratio. NOVA adopts $R = 4$ because it sits within the range validated by Qwen3-Next and Kimi Linear at production scale, not because $R = 4$ has been shown to be uniquely optimal across all settings.

7.3 Window Configuration and Persistent Token Bias

IRIS always uses the full configured window W (no further local/full stratification). Additionally, IRIS receives a persistent token bias from the N_p persistent tokens prepended by LUMEN, which function as a learned attention sink that stabilises the full-window computation across diverse sequence types.

8 Dimension Selection Methodology

NOVA is not a model — it is an architectural method. Any instantiation of NOVA, from a compact research model to a frontier production system, is fully determined by six derivation rules applied to a single input: a compute budget, a parameter budget, or a deployment constraint. This section provides those rules.

8.1 Rule 1 — Base Dimension d from Parameter Budget

Each Nova Cell has two parameter components. Per layer, the total parameter count is approximately:

FLUX: $3d^2$ (projections) + $1.5d^2$ (MLP) + d^2 (output) + $0.03d^2$ (gates) $\approx 5.5d^2$

REASON: $2 \times (d \times d_r) + d_r \times d$ [SwiGLU: 2 input + 1 output] $\approx 8.0d^2$

IRIS: $\sim 4d^2$ per IRIS layer; averaged over 4 layers $\approx 1d^2$ per cell

Total per cell: $\approx 14.5d^2$ **(**$\approx 15d^2$**)**

Iterative dimension selection from parameter budget P_{target}.

$$N_L \leftarrow 8$$

repeat:

$$d \leftarrow \text{round\_to\_multiple\_of\_64\_down}\left(\sqrt{\frac{P_{target}}{15 \cdot N_L}}\right) \tag{48}$$

$$N_L \leftarrow \max\left(8,\ \text{round}\left(\log_2\left(\frac{d}{32}\right)\right)\right)$$

until convergence [typically 2–3 iterations]

Implementation constraint: N_L must be a multiple of 4. This is not a stylistic preference but a mathematical requirement of the 3:1 hybrid ratio. IRIS layers occupy every index $i \equiv 0 \pmod 4$; with $N_L = 4k$ this yields exactly k IRIS layers and $3k$ FLUX layers — the specified ratio.

Any N_L that is not a multiple of 4 produces an irregular terminal group in which the final cluster deviates from 3:1. In particular, $N_L \equiv 1 \pmod 4$ places an IRIS layer at the very last position. This terminal IRIS placement is architecturally harmful: the last layer receives gradient signal directly adjacent to the output projection head, amplifying IRIS gradients relative to FLUX by a factor that can exceed 10× in practice (since gradient magnitude decays approximately exponentially with depth from the loss, and an IRIS layer at position $N_L - 1$ receives the undecayed signal). The starting value $N_L = 8$ in the iterative procedure above satisfies the multiple-of-4 constraint by construction. For larger model scales where the iterative procedure converges to a value other than 8, the result should be rounded to the nearest multiple of 4 before use; rounding to the nearest lower multiple of 4 is preferred when the parameter budget is a hard constraint.

8.2 Rule 2 — Aspect Ratio Adjustment by Task Profile

Task profile	**Recommended adjustment**	**Rationale**
Reasoning-heavy (math, code, multi-step)	Increase N_L by +2; decrease d to maintain budget	Depth improves logical decomposition per layer (Merrill & Sabharwal, 2025)
Recall-intensive (retrieval, QA, lookup)	Increase d to next multiple of 64; reduce N_L by 1	Wider FLUX state (d^2/H) = more associative capacity per layer
General language modelling (balanced)	No adjustment from Rule 1	Kaplan et al.: quality varies <3% across 40× aspect ratio range

Table 7: Aspect ratio adjustment guidance by task profile.

8.3 Rule 3 — Head Dimension

The head dimension $d_h = 64$ is fixed at all model scales. This is the industry standard validated from 70M to 400B parameters across GPT-4, Llama 3, Gemma 3, Qwen3, and Kimi Linear. The FLUX matrix state $S \in \mathbb{R}^{64\times64} = 4\,096$ values per head fits within a single GPU warp, maximising hardware efficiency. The number of heads follows: $H = d/64$.

8.4 Rule 4 — REASON Hidden Dimension

$$d_r = \text{round}\left(\frac{\frac{8}{3} \times d}{256}\right) \times 256 \tag{49}$$

This is the SwiGLU equivalent of a $4d$ expansion factor, rounded to a multiple of 256 for hardware alignment. Examples: $d = 512 \to d_r = 1280$; $d = 768 \to d_r = 2048$; $d = 1024 \to d_r = 2816$.

8.5 Rule 5 — IRIS Window, RoPE Base, and Persistent Tokens

Table 8 sets the IRIS window, RoPE base frequency, and persistent token count as a function of the target context length.

Target context T_{target}	**IRIS window W**	**RoPE base θ_{base}**	**Persistent tokens N_p**
$\leq$ 4K	512	10 000 (default)	4
4K – 64K	1 024	500 000	4
64K – 1M	2 048	5 000 000	8
$>$ 1M	4 096	50 000 000 (YaRN)	16

Table 8: IRIS window, RoPE base frequency, and persistent token count by target context length.

8.6 Rule 6 — FLUX Neural Memory Hidden Dimension

The neural memory MLP inside FLUX uses a hidden dimension $d_m = d/2$. A universal approximator for d-dimensional associations requires no more than $d/2$ hidden units in the regime of interest; increasing beyond this provides diminishing returns at significant parameter cost.

9 Training Dynamics and Auxiliary Losses

9.1 Primary Objective

Standard cross-entropy next-token prediction with label smoothing $\varepsilon = 0.05$:

$$L_{CE} = -\sum_t y_t \cdot \log p_\theta(x_t \mid x_{<t}). \tag{50}$$

9.2 Auxiliary Losses

Loss	**Formula**	**Coeff.**	**Purpose**
FLUX Delta Alignment	$\mathbb{E}_{t,h}\left[\, \|S_{th} \cdot k_{th} - v_{th}\|^2 / d_h \,\right]$	$\lambda \approx 0.010$	Trains FLUX fast path as a genuine key-value store
MNEMOS Calibration	$(\mathbb{E}[s_t] - 0.3)^2$	$\lambda \approx 0.005$	Keeps mean surprise near target; prevents memory from stagnating or thrashing
Relational Emergence	$\left(\|x^R\|^2 / (\|x^D\|^2 + \varepsilon) - 1\right)^2$	$\lambda \approx 0.005$	Enforces role separation: Relational subspace must match Directional scale
REASON Efficiency	$\mathbb{E}_t[\, p_2(t) \,]$	$\lambda \approx 0.003$	Encourages use of Pass 1 only when sufficient
FLUX Gate Calibration	$(\gamma - 0.5)^2$	$\lambda \approx 0.001$	Prevents γ from collapsing before memory specialisation occurs

Table 9: Auxiliary training losses used by NOVA, their formulae, coefficients, and purpose.

Loss activation curriculum and the gamma symmetry constraint. Not all auxiliary losses should be activated simultaneously. Delta alignment, relational emergence, and MNEMOS calibration losses require the model to have developed sufficient representational structure before the loss can provide meaningful gradient signal; activating them at step 0 introduces noise without benefit.

However, the FLUX gate diversity loss (penalising low variance of γ across layers) is recommended, but should be activated from the first optimiser step after warmup ends, not from step 0. During warmup, γ is held fixed to 0 and `log_gamma` is not in the forward computation graph; any loss applied before warmup ends has no gradient effect. Note also that this gate diversity loss is an addition to the gate calibration loss $(\gamma - 0.5)^2$ in the table: the two are complementary (calibration prevents saturation; diversity ensures layer specialisation).

The reason is a structural one. If the gate γ is initialised to `log_gamma` $= 0$ for all FLUX layers, sigmoid(0) $= 0.5$ uniformly, and the gradient of $\mathrm{Var}(\gamma)$ with respect to each γ_i equals $2(\gamma_i - \mathrm{mean}(\gamma))/N = 0$: the diversity loss is structurally at a saddle point and has zero gradient regardless of its coefficient. NOVA breaks this symmetry by initialising `log_gamma` deterministically as a monotonic function of layer depth among the FLUX layers:

$$\texttt{log\_gamma}_i \leftarrow \log\left(\frac{i+1}{N_{FLUX}+1}\right), \qquad i = 0, 1, \ldots, N_{FLUX} - 1, \tag{51}$$

where i is the layer's rank among FLUX layers ($i = 0$ for the shallowest FLUX layer). For $N_{FLUX} = 18$ (the FLUX layer count at $N_L = 24$ under the 3:1 ratio), this produces `log_gamma` ranging from $\log(1/19) \approx -2.944$ at $i = 0$ to $\log(18/19) \approx -0.054$ at $i = 17$, giving an initial $\gamma = \mathrm{sigmoid}(\texttt{log\_gamma})$ spread from approximately 0.050 to 0.486 — monotonically increasing with depth, and with a non-zero gradient of $\mathrm{Var}(\gamma)$ at every layer from the first optimiser step. This is a deliberate trade-off worth stating plainly: because $(i+1) \leq N_{FLUX} < N_{FLUX} + 1$ for every i in range, $\texttt{log\_gamma}_i$ is strictly negative and γ_i is strictly below 0.5 for all layers at initialisation — the monotonic scheme breaks symmetry more strongly than the centred alternative $\texttt{log\_gamma}_i \leftarrow (\mathrm{rank}_i - N_{FLUX}/2) \times 0.18$ (which spreads γ symmetrically around 0.5, e.g. $[0.39, 0.61]$), at the cost of starting every layer on the same side of the FLUX gate calibration loss target $(\gamma - 0.5)^2$. In practice this means the calibration loss provides a consistent early corrective pull toward 0.5 for all layers simultaneously, while the diversity loss and ongoing training separate them by depth; the two objectives are not in conflict but do pull in a more coordinated initial direction than under the centred scheme. Either initialisation removes the saddle point; the monotonic form is preferred when a consistent depth-ordering of fast/slow memory specialisation (shallow layers favouring the fast path, deeper layers more open to the slow path) is a desirable prior, since it seeds that ordering directly rather than leaving it to symmetric random divergence.

In addition to the five core objectives listed above, the training implementation applies a set of module-level regularisation losses: memory stability (constraining θ_M drift to a bounded range), multi-scale anti-repetition penalty (n-gram scales 2–4), recent-token penalty, logit stability (Z-loss), entropy regularisation, RSN gain balance, LayerScale balance, and FLUX gate diversity. These losses do not modify the architecture and are not derived from a single theoretical argument; they address empirically observed failure modes (memory well formation, repetitive generation, logit explosion, gate collapse). Their coefficients are dynamically adjusted throughout training by the adaptive controller described in Section 9.5.

9.3 Optimiser Configuration

AdamW with four parameter groups:

- **Standard group** (full weight decay): all weight matrices of rank ≥ 2 not listed below.
- **No-decay group**: biases, LayerScale scalars (LS_1, LS_2), RSN gain vectors, LUMEN persistent tokens, FLUX gate γ.

- **FLUX gates** (α, β projections): weight decay = 0, LR = 0.5× global. Prevents premature forgetting gate saturation.
- **FLUX neural memory** (W_1, W_2): weight decay = 0.005×global, LR = 0.5×. Allows specialisation without over-regularisation.
- **REASON halting gate** (w_{halt}): LR = 0.1×. Prevents gate from committing before sufficient signal.

9.4 Learning Rate Schedule

Cosine decay with 1% linear warmup of total training steps. Minimum LR at cooldown: 5% of peak. During warmup, REASON operates with p_2 fixed to 0 (Pass 1 only) and FLUX γ fixed to 0 (fast path only). Both are released at warmup end. This staged release prevents early training from over-specialising either module before the full representational structure has stabilised.

9.5 AdaptHP — Adaptive Hyperparameter Control

Training NOVA involves a large number of auxiliary objectives (Section 9.2) whose optimal coefficients change throughout training as each module progressively specialises. Static coefficients are insufficient: the appropriate weight for the memory stability penalty during memory-consolidation phases differs from its value during stable learning. AdaptHP is a training-time controller that adjusts two quantities — the learning rate and the auxiliary loss coefficients — in response to ongoing diagnostics, without manual intervention.

Degradation signal and graduated LR response. The primary diagnostic input is the ratio $R = CE_{current}/CE_{best}$, which measures the current loss relative to the best seen so far. When $R \approx 1.0$, training is progressing normally. When R exceeds 1.3, the model is degrading relative to its historical best and corrective action is applied. AdaptHP maps R to three response levels with a 50-step minimum interval between consecutive actions; no action is taken during warmup or within a post-restart cooldown period.

$R = CE/CE_{best}$	**Response level**	**LR factor**	**Lambda action**
$R < 1.3$	Surveillance	—	Micro-adjustments to unprotected λ only
$1.3 \leq R < 1.6$	Soft correction	×0.92	Dominant unprotected λ reduced by 8%
$1.6 \leq R < 2.0$	Moderate correction	×0.82	Dominant unprotected λ reduced by 20%
$R \geq 2.0$	Strong correction	×0.62	Top-2 unprotected λ each reduced by 30%

Table 10: AdaptHP graduated response levels as a function of the degradation ratio R.

Dynamic lambda modulation. In addition to LR adjustments, AdaptHP monitors module-level weight diagnostics at fixed analysis intervals and modulates individual auxiliary loss coefficients in response to specific pathologies, independently of the R-based LR response.

Diagnostic condition	Lambda modulated	Boost / effect
θ_M drift > 0.92 (memory saturation)	`lam_mem_stable`	$\times 1.15$ per detection; prevents semantic well formation
Stuck steps > 80 (optimisation plateau)	`lam_rep`	$\times 1.10$ per detection; capped at $2\times$ initialisation
γ variance < 0.010 (FLUX gate collapse)	`lam_gamma_div`	$\times 1.20$ per detection; restores FLUX gate layer diversity
Entropy loss > 0.05 (logit entropy spike)	`lam_entropy_reg`	$\times 1.15$ per detection; counters output distribution collapse
$LS_{2,min} < 0.012$ (LayerScale collapse risk)	`lam_ls_balance`	$\times 1.20$ per detection; preserves LayerScale balance

Table 11: Dynamic lambda modulation triggers and their effects.

All boosts are bounded by per-coefficient upper limits ($5\times$ initialisation value, or $2\times$ for `lam_rep`) and are applied only when the corresponding lambda is currently active under the training curriculum (Section 9.2). Modifying an inactive lambda has no gradient effect and would only misrepresent the training log.

Impact tracking and escalation. Each LR restart action is evaluated 30 optimiser steps after it is applied. If the cross-entropy has decreased by at least 0.05 from its pre-action value, the action is recorded as successful and the escalation level is decremented. If no improvement is observed, the escalation level is incremented, causing the next corrective action to apply the next higher response level regardless of R. This prevents successive soft corrections from failing to address a genuine degradation, while avoiding over-correction on transient loss spikes.

LR recovery. Successive LR reductions compound with the cosine decay schedule and can drive the effective learning rate below 6% of peak. At this point the model is effectively frozen and CE may rise sharply. AdaptHP detects this condition (LR $< 6\%$ of peak and $R > 1.2$) and recovers by restoring the LR to 18% of peak with a 400-step cooldown. Critically, both the optimiser per-group LR and the scheduler's `base_lr` are updated simultaneously: without modifying the `base_lr`, the corrected value is overwritten at the next scheduler step and the recovery has no lasting effect.

Protected lambdas. A subset of coefficients is protected from downward revision by graduated LR actions: `lam_rep`, `lam_recent`, `lam_mem_stable`, `lam_rsn_balance`, `lam_mile`, `lam_z_loss`, and `lam_ls_balance`. These govern fundamental training stability properties — anti-repetition, memory norm bounds, logit partition stability, and normalisation balance. Reducing them in response to a degradation signal would compound rather than address the underlying pathology.

9.6 Progressive IRIS Integration and Gradient Balance

IRIS layers compute full-window attention with complexity $O(T \times W \times d)$ per layer. The antisymmetric component introduces an additional gradient pathway through $W_A - W_A^T$, which receives gradients from two paths simultaneously (the direct and transposed contributions), resulting in gradient norms approximately 1.5 to $2\times$ those of a standard projection matrix of equivalent size. Combined with the $T \times W$ pairwise softmax computation, IRIS parameters receive gradient magnitudes that structurally exceed those of FLUX parameters by a factor that can reach several hundred

at full scale. If both modules train simultaneously from the start at equal scale, IRIS dominates the gradient flow and FLUX fails to specialise its recurrent memory.

Recommended approach: progressive signal scaling with gradient normalisation. Apply a scalar multiplier $s \in [0, 1]$ to the IRIS output before the residual addition: $x \leftarrow x + LS_1 \times s \times \text{IRIS}(\text{RSN}(x))$. Initialise s to a small value (e.g. 0.005) and ramp it to 1.0 over approximately 10–15% of total training steps using a smoothstep schedule. During the initial phase where $s < 0.05$, the IRIS output contribution to the residual stream is negligible; at this stage, the gradient flowing back through $s \times \text{IRIS}(\cdot)$ to the IRIS weight matrices is also suppressed, allowing FLUX to establish its recurrent representations unimpeded.

Gradient normalisation as a balance mechanism. In addition to output scaling, apply explicit gradient normalisation after each backward pass: compute the mean gradient norm per parameter across IRIS mixer parameters and FLUX mixer parameters separately, then rescale the IRIS parameter gradients so that their mean norm does not exceed a target ratio (typically 1.0×) relative to the FLUX mean norm. This normalisation decouples the forward contribution of IRIS (controlled by s) from its learning rate (controlled by the gradient magnitude), allowing the two modules to be tuned independently. Once IRIS reaches its target scale, the gradient balance is maintained automatically through the normalisation mechanism and the IRIS signal multiplier can be held at 1.0.

Conditioning IRIS ramp on FLUX stability. The IRIS signal multiplier s should increase only when FLUX is actively learning, not at a fixed schedule. A practical criterion: increase s toward its target only when the ratio $R = CE/best\_CE < 1.3$ and the model has not been "stuck" (no improvement in best CE for more than 60 consecutive optimiser steps). When FLUX is in difficulty, freeze s at its current value and allow FLUX to recover before resuming IRIS integration. This ensures that IRIS always follows FLUX rather than leading it, which is the correct training-time analogue of their inference-time relationship: FLUX handles the sequential context, IRIS provides periodic global corrections.

9.7 Numerical Precision

FLUX matrix state S: float32. FLUX neural memory weights θ_M (runtime copy): float32. All forward computation: bfloat16 via autocast. Loss accumulation: float32. This precision profile minimises the memory overhead of float32 to the two components that genuinely require it (the recurrent state and the memory weights), while maintaining standard training efficiency for all other operations.

10 Architectural Rationale and Expected Advantages Over GPT-Class Models

This section synthesises the quantitative analysis developed throughout this document into a structured comparison. Two parts of this comparison rest on exact arithmetic and algebraic facts established earlier: the FLOP and memory counts (Sections 5.3 and 5.5) are direct derivations from the operation counts of each architecture, and the antisymmetry of A_{rel} (Property 2.2) is a guaranteed consequence of its construction. The remaining claims — that these properties translate into better trained-model quality, and that NOVA's test-time memory update yields genuinely useful online learning — are architectural hypotheses motivated by the design and by related published results

(Titans, Gated DeltaNet, Qwen3-Next, Kimi Linear), not yet first-principles proofs of superiority. Like any new architecture, NOVA's practical advantages need to be confirmed by training and evaluating real models, the same standard every cited prior work was held to before publication.

10.1 Computational Complexity Analysis

Dimension	**GPT-class Transformer**	**NOVA**
Attention computation	$O(T^2 \cdot d)$ per layer	$O(T{\cdot}d^2)$ FLUX fast + $O(T{\cdot}W{\cdot}d)$ IRIS
Context state size (memory)	$O(T{\cdot}d{\cdot}N_L)$ — grows with context	$O(d^2/H \cdot N_L)$ — CONSTANT w.r.t. T
Maximum context (8×H100, 640GB)	∼200K tokens (KV cache limited)	Theoretically unlimited (6 MB state)
FLOPs at T=64K (per layer, d=1024)	1.92×10^{13}	9.02×10^{11} (21× fewer)
FLOPs at T=1M (per layer, d=1024)	4.53×10^{15}	9.02×10^{11} (5 021× fewer)
Scaling class as $T \to \infty$	Quadratic $O(T^2)$	Linear $O(T)$ — fundamentally different regime

Table 12: Computational complexity comparison between GPT-class Transformers and NOVA.

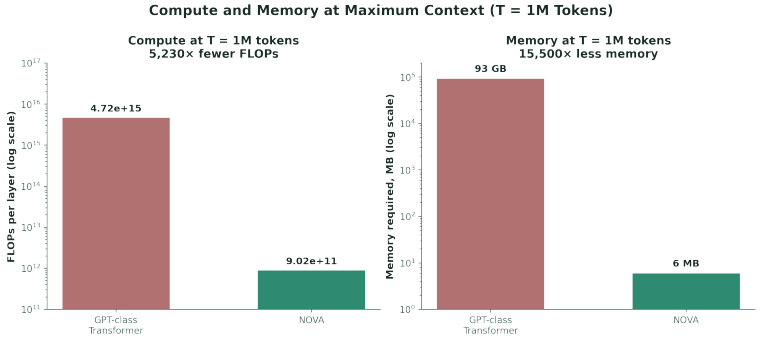

Figure 8: Compute and Memory at Maximum Context (T = 1M) — NOVA requires 5,021× fewer FLOPs and 16,384× less memory than a dense GPT-class Transformer.

At maximum context, the two advantages compound: NOVA's compute and memory requirements are both bounded by exact arithmetic shown earlier (Sections 5.3 and 5.5), not estimates. The underlying scaling class difference is categorical, not quantitative. GPT-class models are permanently bounded by the T^2 wall: however fast the hardware becomes, a doubling of context always quadruples the computational cost. NOVA's $O(T)$ scaling means that a doubling of context doubles the cost — the same relationship as simply adding more tokens to a batch, with no architectural penalty.

10.2 Relational Expressiveness: What the Antisymmetric Component Adds

Observation 10.1 — NOVA's Attention Function Class Contains GPT's as a Special Case. Setting $\lambda_{rel} = 0$ reduces IRIS exactly to standard scaled dot-product attention. The set of functions IRIS can express is therefore a superset of what standard attention expresses, by construction. This containment relationship is real and exact, but it is worth being precise about its weight as evidence: adding a learned, zero-able term to any function generically produces a superset of what that function alone expresses. This is true of nearly every architectural addition proposed in the literature (gating, mixture-of-experts branches, auxiliary heads) and does not by itself establish that the added term will be used productively by a trained model, nor that it improves any benchmark. The meaningful question — whether the antisymmetric term is learned into a useful directional signal in practice, and whether it improves directional reasoning tasks (causal chains, subject-object asymmetry, hierarchical dependencies) — is an empirical one, to be settled by training and evaluating models, not by the containment argument alone.

What the design does provide, independent of training outcomes, is a dedicated and guaranteed-by-construction mechanism for directional asymmetry (Property 2.2) — something standard attention has no equivalent of. Whether that translates into measurably better directional reasoning is the kind of question the field has answered before for comparable additions (e.g. relative position biases, ALiBi, rotary embeddings each needed empirical validation after being proposed), and NOVA's antisymmetric component should be held to the same standard.

10.3 Memory Adaptivity: What Test-Time Updates Add

Observation 10.2 — A Mechanism for Cross-Context Composition Unavailable to Frozen Weights. A GPT-class model with frozen post-training weights can only hold a novel association encountered at inference time in its KV cache — it cannot consolidate that association into its weights, so it cannot compose the new association with prior knowledge through the same pathway the FFN uses for pretrained knowledge. NOVA's FLUX slow path, via the surprise-score-guided test-time gradient update (Section 5.1), updates W_1 and W_2 during inference, giving the model a mechanism — absent in frozen-weight architectures — for writing new associations into the same memory store used for pretrained knowledge. This mechanism exists by construction; whether it reliably improves out-of-distribution associative tasks in a trained model, and at what cost in stability or interference with pretrained knowledge, are empirical questions current evidence (Titans, NeurIPS 2025) is encouraging on but does not settle for NOVA specifically. This is the central empirical claim that pretraining NOVA at scale would need to validate.

10.4 Complete Architecture Comparison

Table 13 draws together the capability comparisons developed throughout this document into a single summary.

Capability	GPT-class Transformer	NOVA
Context scaling	$O(T^2)$ — quadratic wall	$O(T)$ — linear, unlimited context
KV state memory	$O(T \cdot d)$ — grows with context	$O(d^2/H)$ — constant
Associative memory	Frozen FFN post-training	FLUX slow path: test-time gradient update
New knowledge at inference	Impossible (frozen weights)	Encoded via surprise score s_t into W_1, W_2
Relational direction	Content-dependent only	Algebraically antisymmetric by construction (Property 2.2)
Role structure	Homogeneous $\mathbb{R}^d$ vectors	4-role RSLA partition with RSN normalisation
Adaptive computation	Fixed depth per token	2-pass REASON with p_2 halting gate
Representations	Symmetric inner product	Symmetric + antisymmetric component (superset by construction, §10.2)
Memory capacity limit	Hard-bounded by T_{max}	Unlimited: FLUX state never grows with T

Table 13: Complete capability comparison between GPT-class Transformers and NOVA.

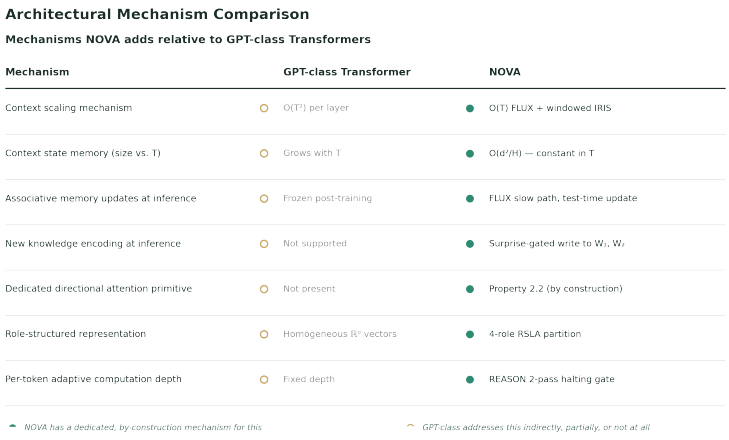

Figure 9: Capability Comparison — NOVA's architectural mechanisms vs. GPT-class Transformers.

11 Conclusion

NOVA is a principled synthesis of three architectural responses to well-documented limitations of GPT-class Transformers: quadratic attention cost, frozen post-training memory, and the absence of a dedicated directional mechanism in the attention score. Each response builds on a real, exact property — an arithmetic FLOP and memory count, an algebraic construction, or a published empirical result from a related architecture — and each is presented in this document with that property clearly distinguished from the separate, open question of whether it improves a trained model in practice.

The $O(T^2)$ computational wall of standard attention is addressed by the FLUX recurrent state, which maintains a constant $O(d^2/H)$ memory representation of the context processed so far — an exact consequence of the recurrence, not an approximation. The frozen-weight memory limitation is addressed by the surprise-score-guided test-time gradient update in the FLUX slow path, a mechanism with no counterpart in frozen-weight architectures, motivated directly by the Titans line of work (NeurIPS 2025). The absence of a dedicated directional attention primitive is addressed by RSLA's algebraically antisymmetric component, which is guaranteed by construction (Property 2.2) to provide a content-independent, sign-reversing directional signal that standard attention has no equivalent of. Whether each mechanism delivers a measurable quality improvement once NOVA is trained at scale is the empirical question this document sets out to motivate, not one it claims to have already answered.

The FLOP arithmetic at $T = 64\text{K}$ shows a $21\times$ computational advantage per layer over a dense GPT-class attention baseline at practical context lengths, growing to $5\,021\times$ at $T = 1\text{M}$ — a difference in scaling class, not merely in constant factors. The FLUX recurrent state requires 6 MB of GPU memory regardless of context length, compared to 103 GB for the KV cache of a comparable dense GPT-class model at $T = 1\text{M}$ tokens. These figures compare against a standard dense-attention baseline without KV-cache compression (e.g. GQA, sliding-window attention); production GPT-class systems deploying such techniques narrow — though do not eliminate — this gap, and a like-for-like comparison against the most heavily optimised contemporary baselines is left for the empirical validation phase.

The 3:1 hybrid pattern — three FLUX layers per IRIS layer — is not an arbitrary internal design choice. It matches the ratio independently arrived at by two major industrial research efforts (Qwen3-Next, Kimi Linear) in 2025, both through large-scale empirical comparison. Other contemporaneous efforts, such as Ling 2.5 (1:7 ratio), have converged on different ratios, and a broader systematic study found accuracy relatively flat across a wide range of ratios. NOVA's choice of $R = 4$ is therefore best understood as a well-validated, production-proven starting point rather than a uniquely determined optimum — and the dimension selection methodology in Section 8 permits it to be revisited as new evidence emerges.

NOVA is a method, not a model. Its dimension selection methodology (Section 8) derives all architectural hyperparameters from a single input (compute budget, parameter budget, or deployment constraint) and produces a fully determined, hardware-aligned architecture at any scale. The components it combines — the RSLA representational substrate, the unified FLUX memory architecture, and the empirically grounded 3:1 hybrid ratio — are each individually well-motivated by exact arithmetic, exact algebraic construction, or published results from related architectures. The next step toward establishing NOVA's advantages, as for any new architecture, is training and evaluating models at meaningful scale against strong contemporary baselines.

12 References

Behrouz, A., Zhong, P., & Mirrokni, V. (2025). Titans: Learning to memorize at test time. In *Advances in Neural Information Processing Systems 38 (NeurIPS 2025)*. arXiv:2501.00663.

Geva, M., Schuster, R., Berant, J., & Levy, O. (2021). Transformer feed-forward layers are key-value memories. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing (EMNLP 2021)* (pp. 5484–5495). Association for Computational Linguistics.

Kaplan, J., McCandlish, S., Henighan, T., Brown, T. B., Chess, B., Child, R., Gray, S., Radford, A., Wu, J., & Amodei, D. (2020). Scaling laws for neural language models. arXiv:2001.08361.

Li, Z., et al. (2025). Kimi Linear: An expressive, efficient attention architecture. arXiv:2510.26692.

Merrill, W., & Sabharwal, A. (2025). A little depth goes a long way: The expressive power of log-depth transformers. arXiv:2503.03961.

Qwen Team, Alibaba Group. (2025). Qwen3-Next: A new generation of ultra-efficient model architecture [Technical blog post]. Alibaba Cloud / Qwen.

Saxe, A. M., McClelland, J. L., & Ganguli, S. (2014). Exact solutions to the nonlinear dynamics of learning in deep linear neural networks. In *Proceedings of the 2nd International Conference on Learning Representations (ICLR 2014)*. arXiv:1312.6120.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is all you need. In *Advances in Neural Information Processing Systems 30 (NeurIPS 2017)* (pp. 5998–6008).

Yang, S., Kautz, J., & Hatamizadeh, A. (2025). Gated Delta Networks: Improving Mamba2 with delta rule. In *Proceedings of the 13th International Conference on Learning Representations (ICLR 2025)*. arXiv:2412.06464.

NOVA — Neural Organic Vivid Architecture
EltagAI — 2026